

DSA PROJECT PROPOSAL

**Phone Book and Call Log Management System using
C++**

Name: Mishal Fahim

Roll No: K240041

Submitted to: Sir Shafique Rehman

Phone Book and Call Log Management System using C++

1. Introduction

The Phone Book and Call Log Management System is a console-based C++ application designed to store, manage, and retrieve contact information efficiently. It also maintains call logs, allowing users to simulate calls, store call duration, and review call history. The system integrates **File Handling**, **Hash Tables (for Contacts)**, and **Linked Lists (for Call Logs)** to allow persistent and efficient data management.

The goal of this project is to provide a simple yet functional phonebook application that performs basic CRUD operations along with call history tracking.

2. Objectives

- I. To develop an interactive phone book application in C++.
- II. To store and retrieve contact information using **hash maps**.
- III. To maintain and display call history using **linked lists**.
- IV. To provide persistent storage using **file handling**.
- V. To demonstrate searching, sorting, and updating operations.
- VI. To understand implementation of data structures in real-world use.

3. System Features

Contact Management (HashTable)

- ✓ Add Contact: Stores new contact after validating phone number.
- ✓ Search Contact: Finds contact using hashing.
- ✓ Update Contact: Edits phone number of an existing contact.
- ✓ Delete Contact: Removes a contact from the hash table.
- ✓ Display All Contacts: Shows all contacts sorted A–Z.

- ✓ Auto Save/Load: Contacts saved in contacts.txt.

Call Log System (Linked List)

- ✓ Make a Call: User enters phone number and duration.
- ✓ Saves Call: Stored in call_history.txt.
- ✓ View History: Shows recent calls first.
- ✓ Sort by Duration: Sorts call logs using bubble sort.

4. Technology & Data Structures Used

- I. Language: C++.
- II. Contact Storage: Hash Table using vector<list<Contact>>.
- III. Call History: Singly Linked List.
- IV. File Handling: fstream (ifstream, ofstream).
- V. Sorting: Bubble Sort for call durations.
- VI. Searching: Hashing for direct access.
- VII. Validation: Phone number format check.

5. Working of the System

Flow of Execution

- I. Load existing data from text files
- II. Display menu repeatedly until the user exits
- III. Perform tasks based on user input
- IV. Data automatically updates in files

6. Code Breakdown (Short Explanation)

Contact Management (Hash Table)

- ✓ Hash function converts contact name → index
- ✓ Handles collisions using **separate chaining (list)**
- ✓ Allows insertion, deletion, searching in O(1) average time

```
int hashFunction(string key) {
```

```

    unsigned long hash = 5381;
    for (char c : key)
        hash = ((hash << 5) + hash) + c;
    return hash % TABLE_SIZE;
}

```

Call History (Linked List)

- ✓ Each call stored as a **node**
- ✓ New calls inserted at head → recent first
- ✓ Sorting swaps node data based on duration

File Handling

- ✓ Two files ensure data is not lost when program closes:
- ✓ contacts.txt: Name, Phone
- ✓ call_history.txt: Number, Name, Type, Timestamp, Duration

7. Sample Output:

----> PHONE BOOK & CALL LOG SYSTEM <----

1. Add Contact
2. Search Contact
3. Delete Contact
4. Display All Contacts
5. Make a Call (Simulate)
6. View Call History
7. Sort History by Duration
8. Update Contact
0. Exit

Enter choice:

8. Advantages:

- I. Fast search and retrieval using hashing
- II. Dynamic call log handling using linked list
- III. Data persists even after closing program
- IV. Easy to use interface
- V. Structured and modular design

9. Limitations & Future Improvements

- ✓ No GUI: Add SFML/Qt interface
- ✓ No incoming call simulation: Add call types & filters
- ✓ No duplicate number check: Add phone-number indexing
- ✓ Contact-only search by name: Add search by number
- ✓ No SMS feature: Add messaging module

10. Conclusion

This project successfully demonstrates a practical implementation of **Data Structures (Hash Table + Linked List)** along with **File Handling** in C++. The system can store, update, delete, and display contacts efficiently, as well as maintain complete call logs. It can be further enhanced to develop a fully functional mobile-like contact manager.